

**CEBU INSTITUTE OF TECHNOLOGY - UNIVERSITY
COLLEGE OF COMPUTER STUDIES**

Software Requirements Specification

For

**RigMD: A Symptom-Guided Diagnostic and Decision Support System
for Non-Technical Windows Desktop PC Users**

Team Code: 2526-sem2-it332-11

Semester 2, AY 2025-2026

Prepared by:

Libato, Michael Grant

Macansantos, Axcelson

Ruperez, Raymart

Maestrado, Ralph Keane

Labaya, Godwin

Change History

Version	Date	Description	Author
1.0	May 2026	Initial draft of the Software Requirements Specification.	Michael Grant S. Libato
1.1	May 22, 2026		Ralph Keane A. Maestrado

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms, and Abbreviations

1.4 References

2. Overall Description

2.1 Product Perspective

2.2 User Characteristics

2.3 Product Functions

2.4 Constraints

2.5 Assumptions and Dependencies

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 Hardware Interfaces

3.1.2 Software Interfaces

3.1.3 Communications Interfaces

3.2 Functional Requirements and Wireframes

Module 1: System Profile and System Overview Module

Module 2: Structured OS Symptom Intake Module

Module 3: Internal Diagnostic Interpretation Engine

Module 4: Advisory Action Module

Module 5: Transparent Result Dashboard and Report Output Module

Module 6: Session History and Recurring Pattern Detection Module

3.3 Data Requirements

3.4 Non-Functional Requirements

4. Validation and Evaluation Requirements

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification is to define the functional, non-functional, interface, data, and evaluation requirements for RigMD: A Symptom-Guided Diagnostic and Decision Support System for Non-Technical Windows Desktop PC Users. The document serves as the technical reference for the development team, academic evaluators, and project stakeholders during the design, development, testing, and evaluation of the prototype for Semester 2, AY 2025-2026.

RigMD is specified as a decision-support desktop application. It does not claim to perform confirmed physical diagnosis or replace qualified hardware inspection. Instead, it helps users interpret Windows-observable PC symptoms, relate those symptoms to their system profile, and decide whether to monitor, maintain, troubleshoot, or escalate the issue for professional inspection.

1.2 Scope

RigMD is a downloadable Windows desktop application intended for individual consumer desktop PC owners who experience Windows-observable performance symptoms. The system guides the user through a session-based workflow: the system profile is created or confirmed, the user answers structured symptom questions, the system interprets the symptom pattern against the profile and prior history, the result dashboard presents an action-classified advisory, and the completed session is saved locally for future review.

The system targets symptoms that can be observed through the Windows environment, such as slow boot, freezing, repeated blue screen errors, driver conflict notifications, S.M.A.R.T. storage health alerts, repeated application crashes, no-display behavior after driver changes, and other OS-level performance or stability indicators.

The system includes the following six core modules:

- System Profile Module - stores or confirms key desktop PC specifications used as diagnostic context, including CPU model, RAM capacity, storage type and capacity, operating system version, GPU and chipset driver versions, and system age.
- Structured OS Symptom Intake Module - guides users through controlled questions that collect consistent symptom data instead of relying on open-ended free text.

- Internal Diagnostic Interpretation Engine - maps system profile data, Windows-observable symptom inputs, and prior session history to probable internal cause categories.
- Advisory Action Module - classifies each result into one of four action categories: Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection.
- Transparent Result Dashboard - presents the advisory result, probable internal cause, confidence label, plain-language explanation, system profile details used, warning signs reference, and recommended next steps.
- Session History and Recurring Pattern Detection Module - stores diagnostic sessions locally and flags repeated symptoms or internal cause categories across two or more sessions.

RigMD is scoped to Windows desktop PCs that are functional or partially functional and can provide observable Windows-level symptoms. Laptops, mobile devices, servers, workstations, and enterprise-managed environments are outside the project scope. Fully non-functional units are limited to a safety-oriented boot-status triage path that recommends professional inspection when the condition exceeds software-layer resolution.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification.
RigMD	The proposed Windows desktop diagnostic decision-support application.
PC	Personal Computer; in this document, it refers to a consumer Windows desktop PC.
AI	Artificial Intelligence.
CPU	Central Processing Unit.
GPU	Graphics Processing Unit.
RAM	Random Access Memory.
PSU	Power Supply Unit.
SSD	Solid-State Drive.
HDD	Hard Disk Drive.
S.M.A.R.T.	Self-Monitoring, Analysis, and Reporting Technology; storage health monitoring data used as an observable warning indicator.

WMI	Windows Management Instrumentation; a Windows system information interface that may be used to assist local system profile detection.
SQLite	A local embedded database used to store system profile and diagnostic session history.
CustomTkinter	A Python GUI framework used for the Windows desktop interface.
Gemini	The AI model API used to generate constrained plain-language explanations for diagnostic results.
SUS	System Usability Scale; a standardized usability evaluation instrument.
MVP	Minimum Viable Product.
NFR	Non-Functional Requirement.
UI	User Interface.
OS	Operating System.
AY	Academic Year.
Windows-observable symptom	A PC issue visible through Windows behavior, such as boot delay, freezing, crash reports, driver warnings, event log errors, S.M.A.R.T. alerts, or repeated application failures.
Diagnostic Session	A single guided workflow in which the user answers structured questions and receives an advisory output.
System Profile	A stored or confirmed record of the user's desktop PC specifications used as context for diagnostic interpretation.
Internal Cause Category	A probable source category identified by the diagnostic engine, such as OS performance degradation, driver conflict, storage health behavior, or boot/startup failure.
Advisory Output	The system-generated result containing the probable internal cause, action category, explanation, confidence label, warning signs, and recommended next steps.

Action Category	The practical next-step classification assigned to a result: Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection.
Recurring Pattern	A repeated symptom type or internal cause category detected across two or more sessions for the same system profile.
Decision Confidence	The user's self-reported confidence in understanding the PC problem, likely cause, recommendation, and next action.

1.4 References

- Dahouk, A. W., & Abu-Naser, S. S. (2018). A proposed knowledge based system for desktop PC troubleshooting. *International Journal of Academic Pedagogical Research*, 2(6), 1–8.
- Ravasio, P., Schär, S. G., & Krueger, H. (2004). In pursuit of desktop evolution: User problems and practices with modern desktop systems. *ACM Transactions on Computer-Human Interaction*, 11(2), 156–180.
- Kostopoulos, G., Davrazos, G., & Kotsiantis, S. (2024). Explainable artificial intelligence-based decision support systems: A recent review. *Electronics*, 13(14), 2842.
- Ravi, M., Negi, A., Bommi, N. S., & Rouf, N. (2025). Evolution of AI-driven decision making with decision support systems, expert systems, recommender systems, and XAI. *IETE Technical Review*, 42(4), 428-465.
- Zhang, W., Yang, D., & Wang, H. (2019). Data-driven methods for predictive maintenance of industrial equipment: A survey. *IEEE Systems Journal*, 13(3), 2213–2227.
- Brooke, J. (1996). SUS: A quick and dirty usability scale. In *Usability Evaluation in Industry* (pp. 189–194). Taylor and Francis.
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.

2. Overall Description

2.1 Product Perspective

RigMD is an independent Windows desktop application. It is not an extension of an existing utility and does not require a cloud database or server deployment. It is positioned as a brand-agnostic decision-support system that consolidates functions commonly scattered across brand-specific diagnostic utilities, standalone monitoring tools, Windows troubleshooters, driver advisory tools, and general-purpose AI tools.

The system is designed around a structured, session-based workflow instead of an open-ended chatbot. Structured intake makes each diagnostic session consistent, testable, and comparable against expert-validated diagnostic scenarios. It also helps non-technical users provide relevant symptom information without needing to know which technical details matter.

The prototype will be implemented as a Python desktop application using the CustomTkinter GUI framework. Diagnostic rule logic will use structured Python mappings between symptom inputs, system profile fields, and defined internal cause categories. Session history will be stored locally using SQLite. AI-assisted explanation generation will use the Gemini API through constrained prompt templates that receive only structured symptom and system profile context.

2.2 User Characteristics

The primary users are individual Windows desktop PC owners who lack the technical background to interpret PC symptoms or raw monitoring data independently. Target users include:

- Students who rely on desktop PCs for academic work and may not know how to diagnose performance problems.
- Gamers who experience slow boot, freezing, stuttering, crashes, driver warnings, or performance degradation.
- Beginner PC builders who need guided support after assembling or configuring a desktop PC.
- Budget desktop users who want to avoid unnecessary service costs or incorrect troubleshooting steps.
- Content creators who depend on desktop PC performance for video editing, rendering, streaming, or heavy application use.
- General home users who need plain-language guidance when their desktop PC becomes unstable or slow.

Users are expected to have basic computer literacy, such as opening a Windows desktop application, selecting options, reading simple explanations, and following guided prompts. They are

not expected to understand hardware diagnostics, event logs, S.M.A.R.T. status, or driver conflict analysis without assistance.

2.3 Product Functions

- Detect or confirm a system profile containing key desktop PC specifications.
- Guide the user through structured Windows-observable symptom questions.
- Classify reported symptoms into supported internal symptom and cause categories.
- Generate a probable advisory result using rule-based mapping and AI-assisted explanation.
- Classify the recommended next action into Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection.
- Present the result through a transparent dashboard with reasoning, profile factors, warning signs, and recommended next steps.
- Save completed sessions locally and display diagnostic history.
- Detect recurring symptoms or recurring internal cause categories across sessions.
- Generate a technician-ready report that summarizes the session, profile snapshot, symptom answers, warning signs, result, and recommendation.

2.4 Constraints

Platform Constraint: The prototype is limited to Windows desktop operating systems. macOS, Linux, mobile devices, laptops, servers, workstations, and enterprise-managed systems are outside the current scope.

Diagnostic Scope Constraint: The full diagnostic workflow applies only to functional or partially functional desktop PCs with Windows-observable symptoms. Fully non-functional units are handled only through a safety-oriented boot-status triage checklist.

System Profile Constraint: The system profile relies on local system information where technically feasible. The current wireframe scope presents system profile data as automatically detected or confirmed by the application; incomplete or low-confidence detection must be flagged clearly to the user.

Decision-Support Constraint: The system produces probable advisory findings, not guaranteed diagnoses. It cannot perform physical inspection, voltage measurement, component substitution, or electrical testing.

No Replacement or Upgrade Recommendation Constraint: The system does not directly recommend component replacement or upgrade purchases. It recommends monitoring, maintenance,

software-layer troubleshooting, or professional inspection when symptoms exceed the supported advisory scope.

Local Storage Constraint: Session history and profile data are stored locally on the user's device using SQLite or equivalent local storage. The prototype does not implement cloud synchronization, user account management, or remote data access.

AI Scope Constraint: AI-assisted explanation generation is constrained by structured symptom input, system profile context, and rule-based diagnostic output. The AI component does not operate as an unconstrained open-ended prompt interface.

Internet Dependency Constraint: Internet access is required only when AI-assisted explanation generation is used. Core diagnostic workflow functions must remain available offline through rule-based logic and fallback explanation templates.

Time Constraint: The system must reach prototype-level completion by the end of Semester 2, AY 2025-2026.

Evaluation Constraint: Expert validation requires at least two hardware-knowledgeable evaluators and at least 20 controlled diagnostic scenarios. Usability evaluation requires at least 10 target users.

2.5 Assumptions and Dependencies

- The target device runs a supported version of Windows and can launch the installed desktop application.
- The application has permission to read required local system information and write local SQLite/session files.
- System profile detection may vary depending on Windows permissions, device configuration, and library support.
- Users answer symptom questions accurately and select the option that best matches the observed behavior.
- AI-assisted explanation generation depends on availability of the Gemini API and a stable HTTPS connection.
- If the AI API is unavailable, the system can still complete the diagnostic workflow using rule-based outputs and fallback explanation templates.
- Expert evaluators have sufficient hardware and Windows troubleshooting knowledge to evaluate diagnostic output relevance and safety.
- Local data persistence depends on available disk space and file system write access.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 Hardware Interfaces

The system interacts with the following hardware and local device resources:

- **Local File System:** Reads and writes application data, system profile records, and diagnostic session history on the user's device.
- **System Information Interfaces:** May query WMI, Python platform data, or supported hardware information libraries to detect CPU, RAM, storage, operating system, driver versions, and related profile values.
- **Display:** Requires a minimum recommended display resolution of 1280 x 720 pixels to render the desktop interface correctly.
- **Input Devices:** Uses standard keyboard and mouse input for navigation, answer selection, search, filters, and export commands.
- **Storage:** Requires local disk space for the installed application, SQLite database, generated reports, and cached session records. Estimated prototype footprint should remain below 500 MB excluding user-exported reports.

3.1.2 Software Interfaces

- **Operating System:** Windows 10 (version 1903 or later), or Windows 11.
- **Application Framework:** Python with the CustomTkinter GUI framework for the desktop interface.
- **Diagnostic Rule Engine:** Python conditional mappings and rule structures that connect symptom responses, profile data, and internal cause categories.
- **Local Database:** SQLite through Python's built-in sqlite3 module for profile and session history storage.
- **System Information Libraries:** Python platform and WMI libraries, or equivalent local libraries, for system profile detection where feasible.
- **AI Explanation API:** Gemini API using constrained prompt templates for plain-language advisory explanations.
- **Report Output Support:** Local export functionality for technician-ready summaries, including copy, print, and PDF export actions as represented in the Reports wireframe.

3.1.3 Communications Interfaces

RigMD requires network communication only for AI-assisted explanation generation. All other major functions, including profile management, symptom intake, rule-based interpretation, advisory classification, result display, session history, recurring pattern detection, and report review, must be available without internet access.

- All AI API communication must use HTTPS.
- The AI API request must use only structured symptom data, profile context, classified result data, and approved prompt templates.
- The system must not transmit personally identifiable information, full local files, or complete local session history to the AI API.
- If the AI API is unavailable, the system must display a user-facing notice and complete the session using rule-based explanation templates.
- No remote database, cloud synchronization, or user account server is required for the prototype.

3.2 Functional Requirements and Wireframes

The following functional requirements are organized according to the six core modules described in the proposal and the wireframes prepared for the SRS. Each module includes requirements and the corresponding wireframe reference.

Module 1: System Profile and System Overview Module

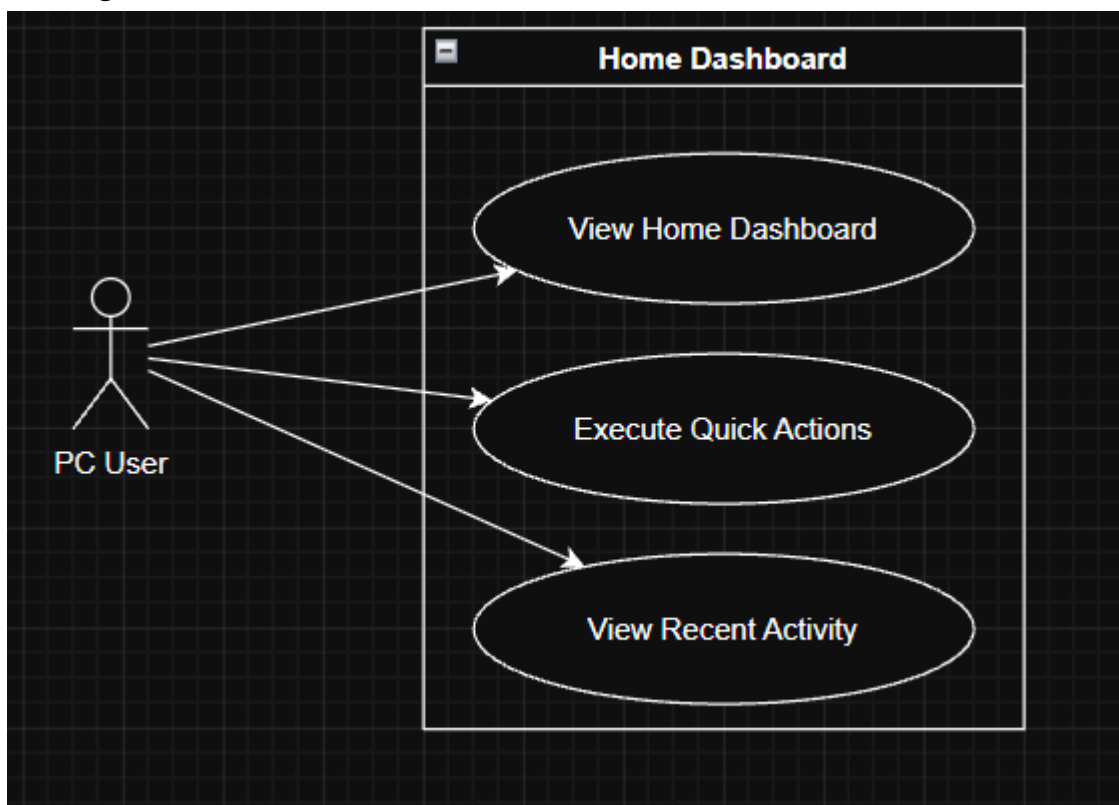
1.1 Home Dashboard & Quick Actions

Use Case Description:

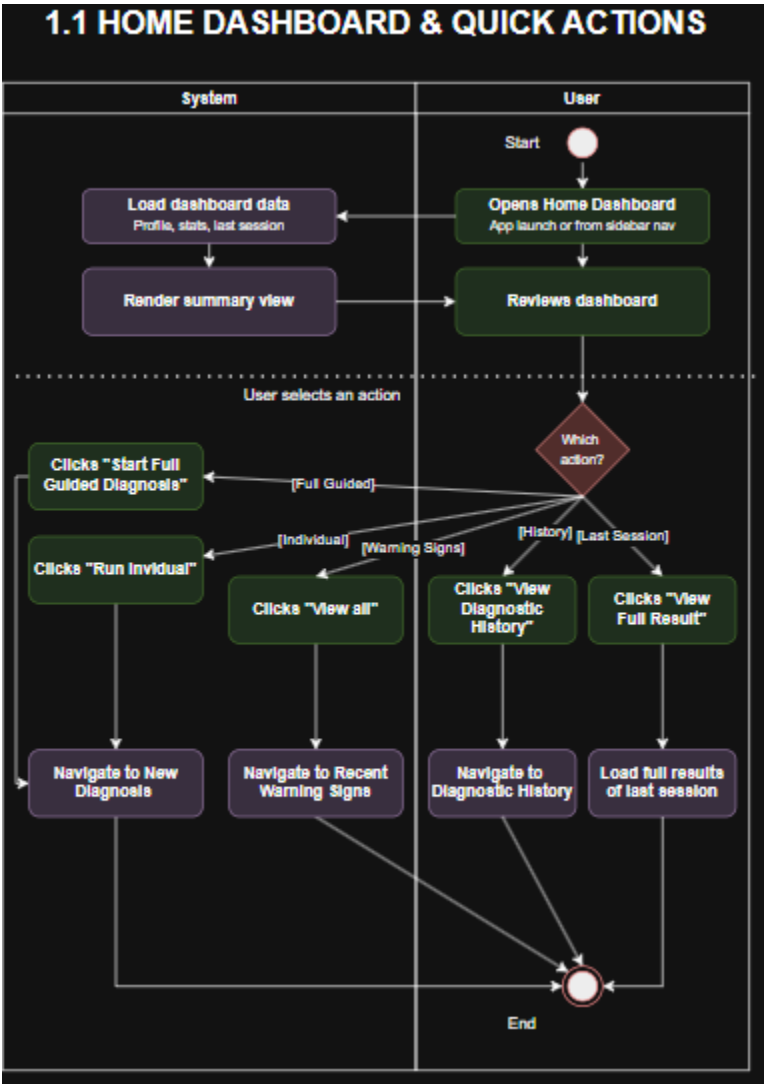
Use Case ID	UC-1.1
Use Case Name	Home Dashboard & Quick Actions
Actor	PC User

Precondition	The application is launched, and the main interface loads.
Main Flow	The user navigates to the Home Dashboard. The system displays a high-level summary of the system's status, recent diagnostic activity trends, and active warning signs. The user executes quick action shortcuts to instantly start a new diagnostic scan or review their history without navigating through multiple menus.
Alternative Flow	If no recent activity exists, the system displays a welcome prompt to start a new scan.
Postcondition	The user selects a quick action or navigates to another module.
Non-functional Requirement	The application shall launch and display the main dashboard within 5 seconds on a device meeting the minimum requirements.

Use Case Diagram



Activity Diagram



Wireframe



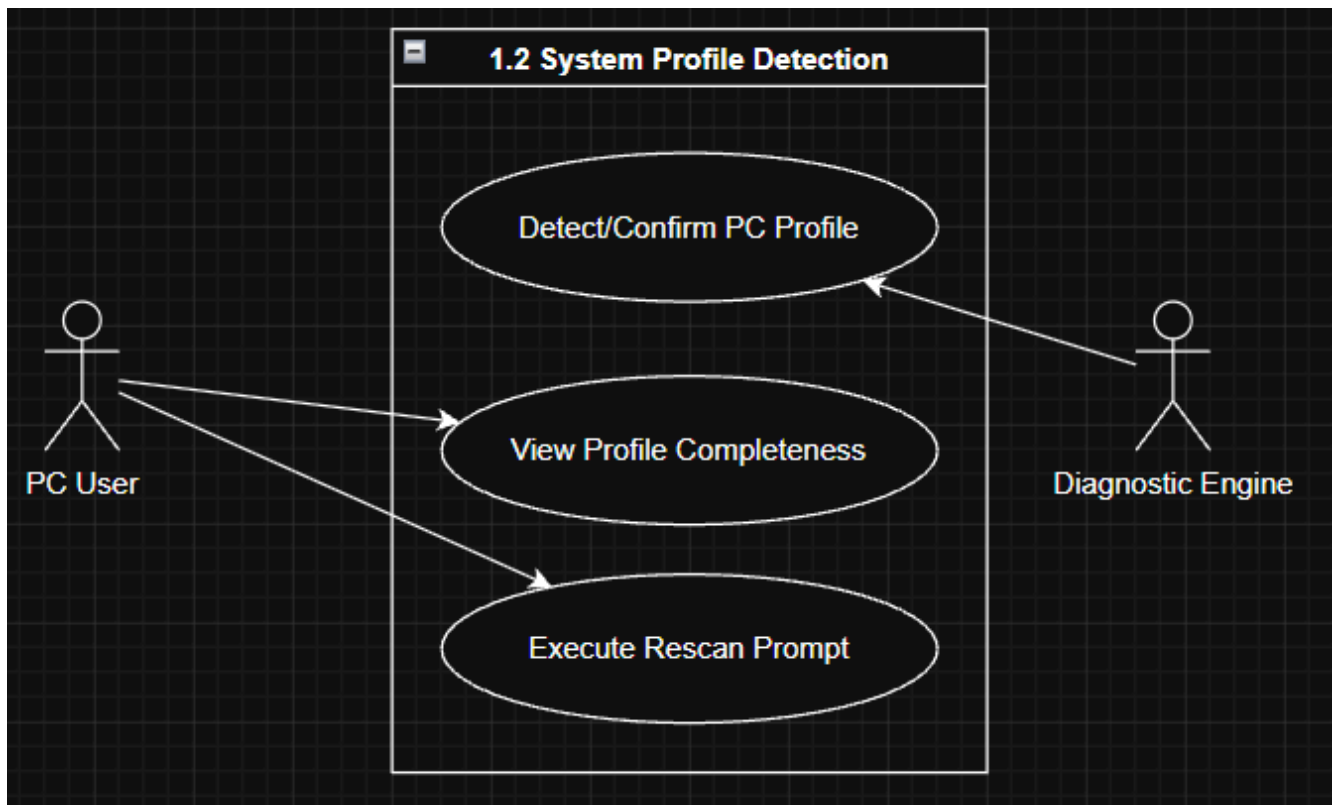
1.2 System Profile Detection

Use Case Description

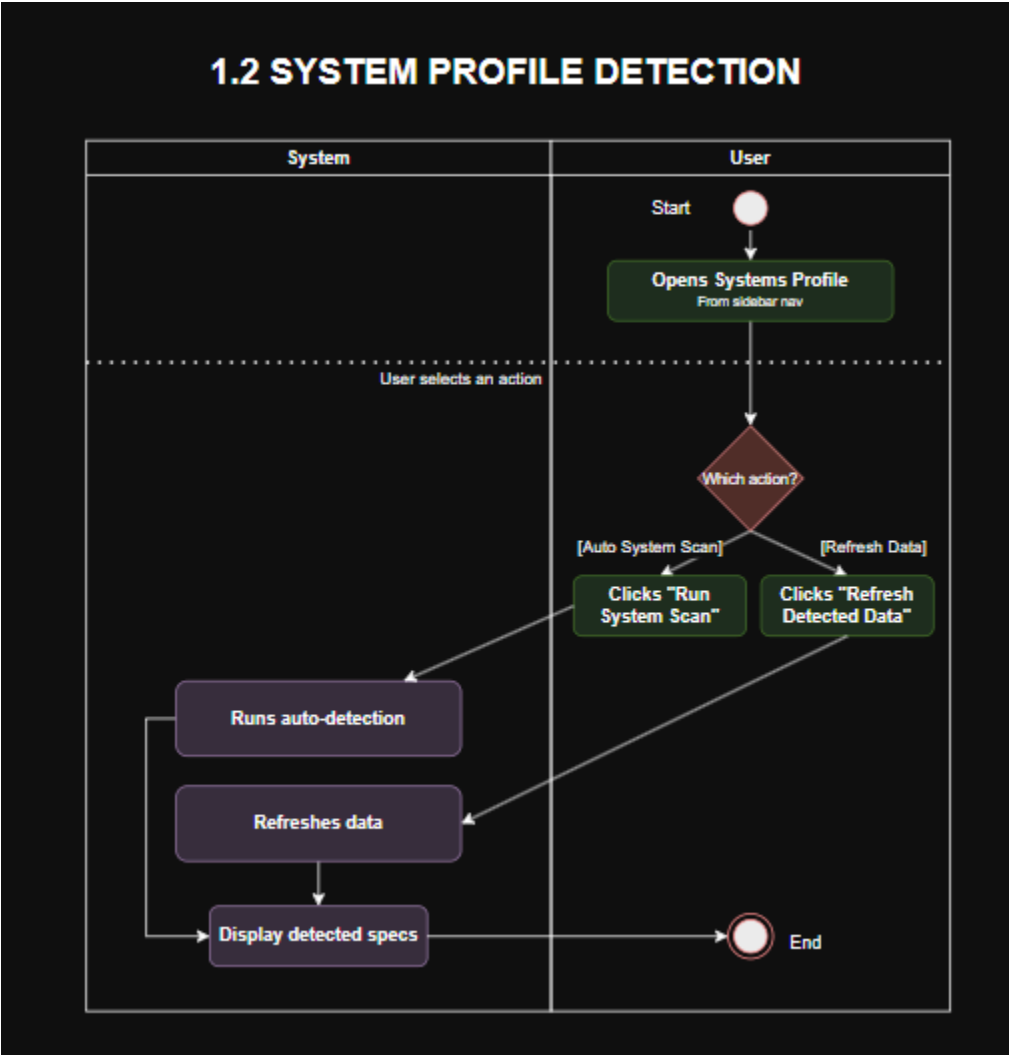
Use Case ID	UC-1.2
Use Case Name	System Profile Detection
Actor	Diagnostic Engine, PC User
Precondition	The application is launched.

Main Flow	The Diagnostic Engine automatically scans and detects the hardware and software specifications of the desktop PC. The user views this profile to verify the completeness and confidence level of the detection. The user executes a rescan prompt if any system data is missing or displays a low-confidence warning since manual editing is disabled.
Alternative Flow	If the system cannot detect certain hardware due to permissions, it flags the missing data and prompts the user to check Windows settings.
Postcondition	The system profile is saved locally and used as context for diagnostics.
Non-functional Requirement	All system profile records shall be stored locally on the user's device.

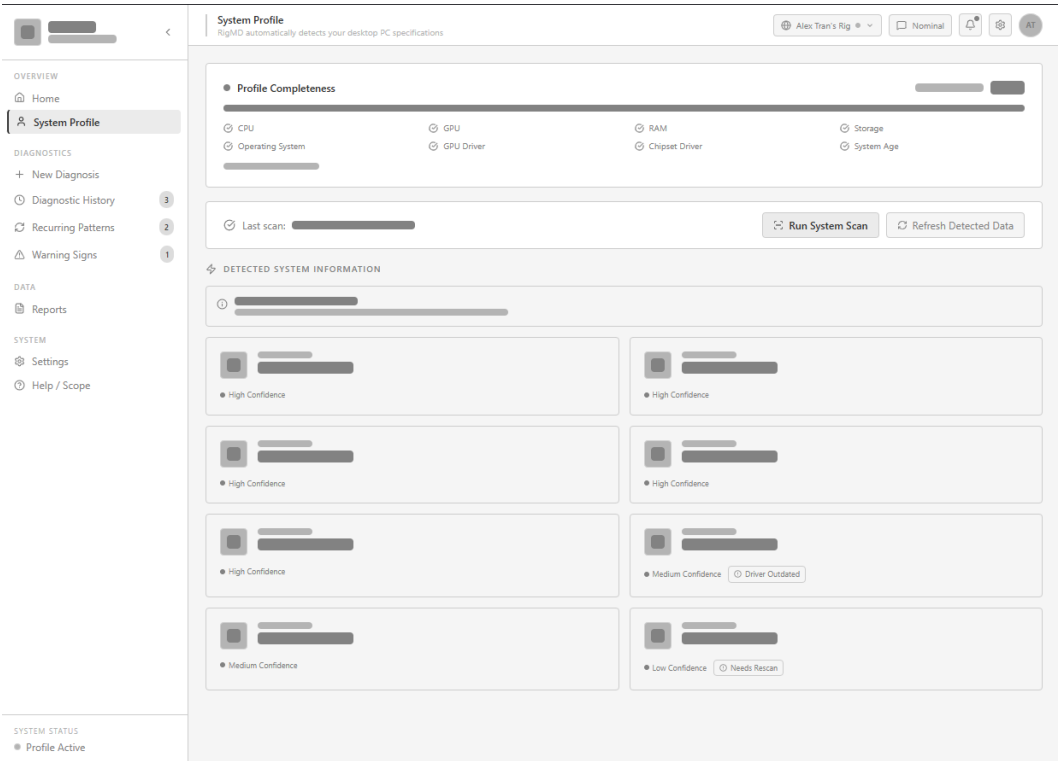
Use Case Diagram



Activity Diagram



Wireframe



Module 2: Structured OS Symptom Intake Module

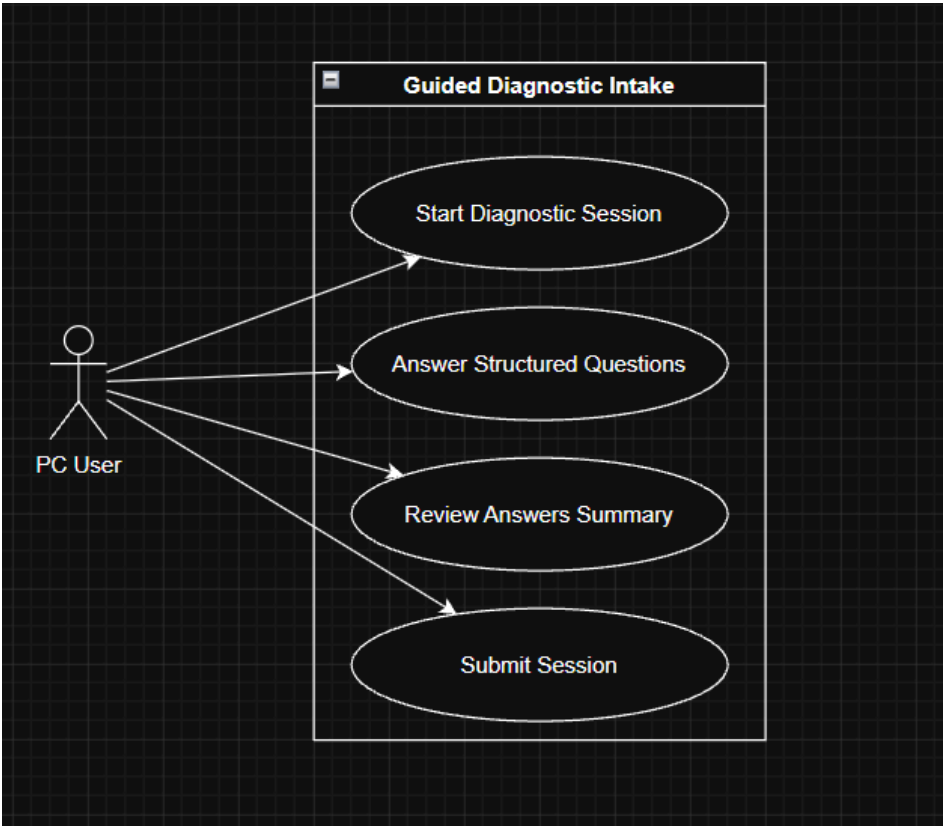
2.1 Guided Diagnostic Intake

Use Case Description

Use Case ID	UC-2.1
Use Case Name	Guided Diagnostic Intake
Actor	PC User

Precondition	The user initiates a new diagnostic session from the dashboard.
Main Flow	The user interacts with the interface to answer a series of controlled questions spanning eight distinct steps to collect diagnostic parameters without free-text entry. The user continuously reviews a running summary of their provided answers. The user navigates backward to make corrections before finalizing and submitting the session.
Alternative Flow	The user can cancel the diagnostic session at any time before submission, discarding the unsaved data.
Postcondition	The structured symptom data is submitted to the Diagnostic Engine for interpretation.
Non-functional Requirement	Each questionnaire step shall transition within 500 milliseconds after a user response is selected.

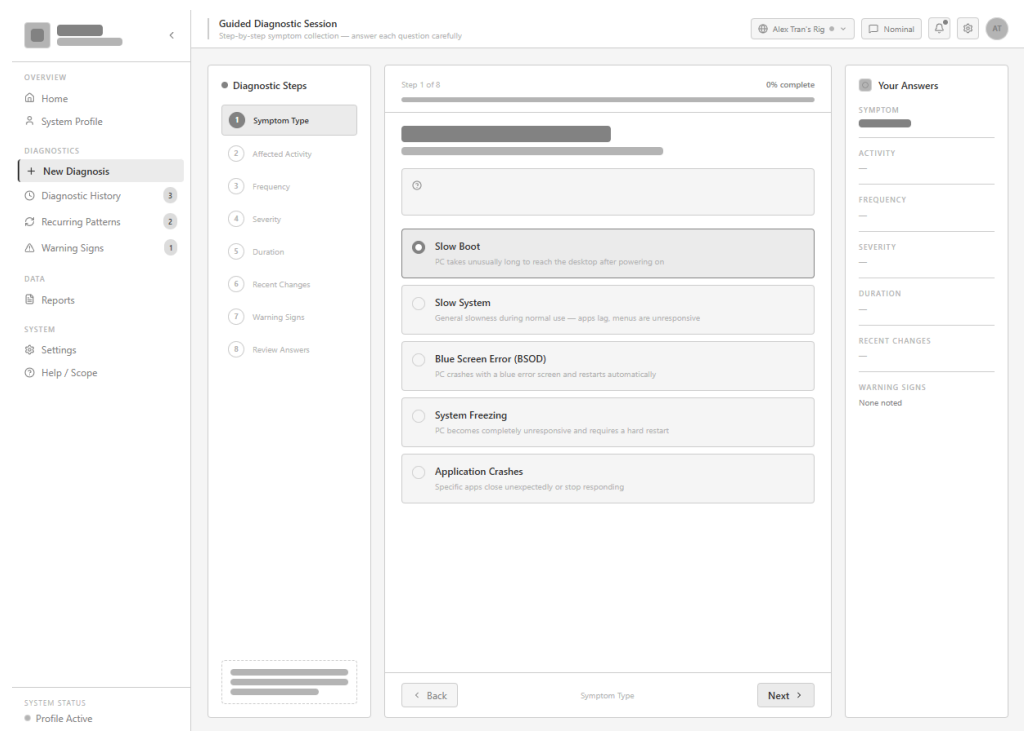
Use Case Diagram



Activity Diagram



Wireframe



Module 3: Internal Diagnostic Interpretation Engine

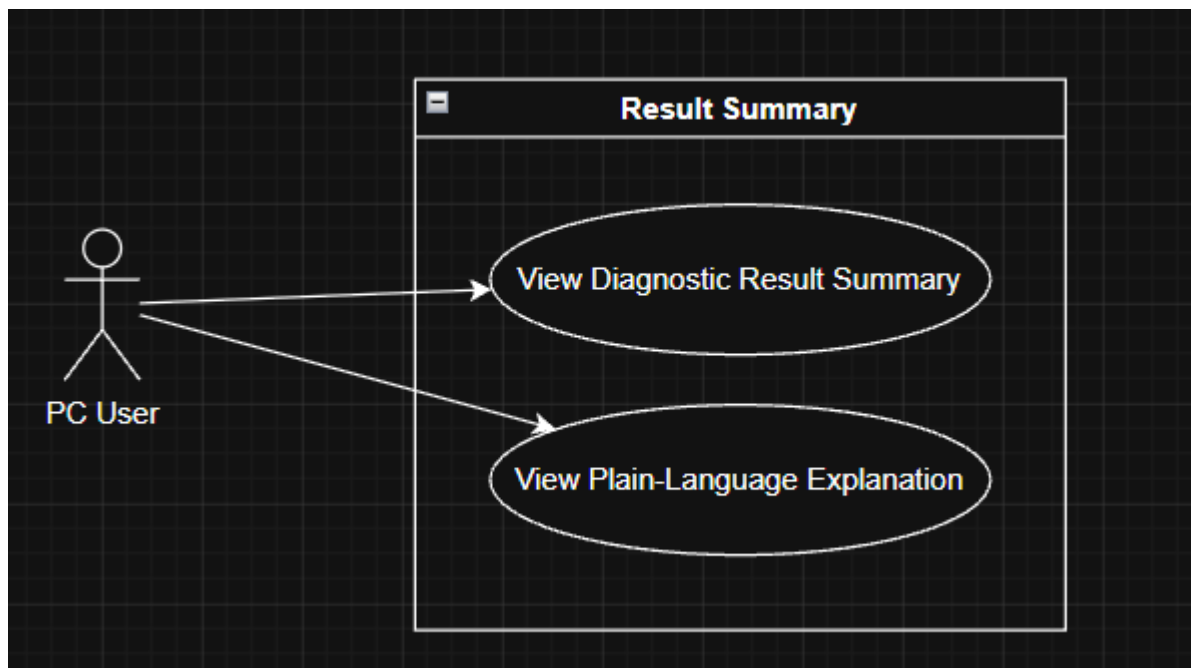
Module 3.1: Result Summary

Use Case Description

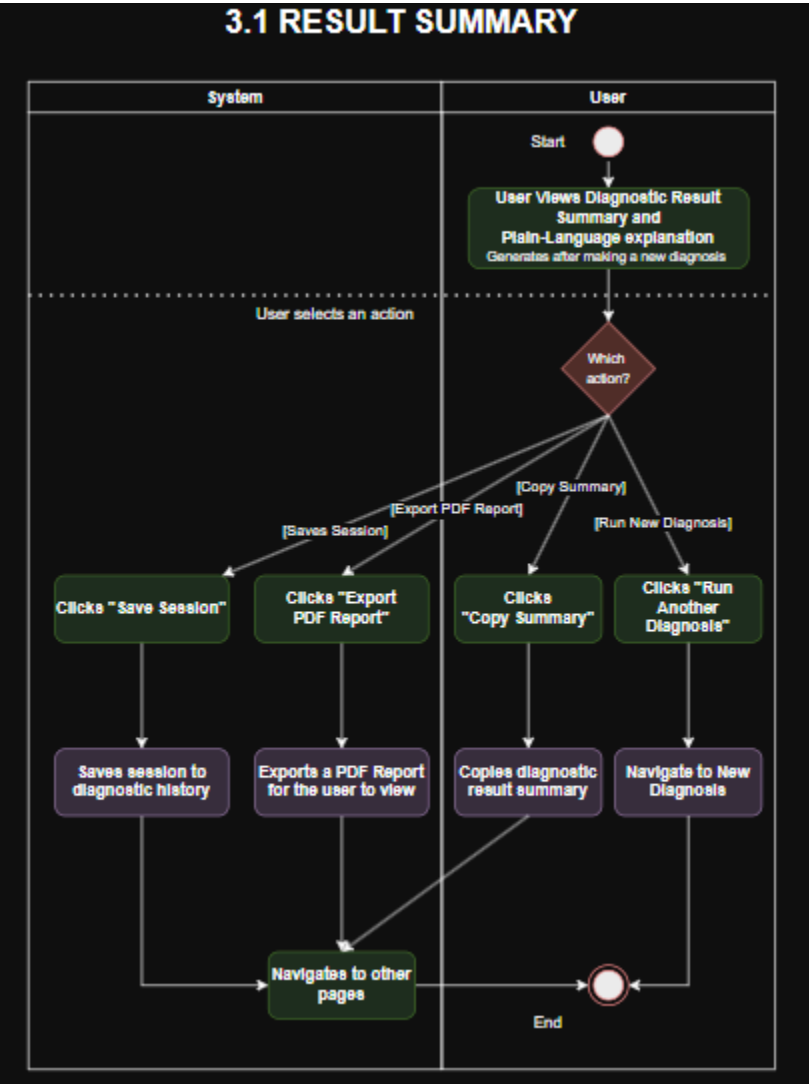
Use Case ID	UC-3.1
Use Case Name	Diagnostic Result Summary Visualization
Actor	PC User
Precondition	The system has successfully generated the diagnostic advisory.

Main Flow	The user navigates to the Diagnostic Result screen. The system displays the probable internal cause, diagnostic confidence, and the primary recommended action. The user reads the plain-language explanation to understand why the specific result was provided based on their reported symptoms.
Alternative Flow	If the explanation cannot be generated dynamically, the system displays a default rule-based summary and informs the user.
Postcondition	The user understands the high-level diagnostic finding.
Non-functional Requirement	The result summary must load and render in under 3 seconds.

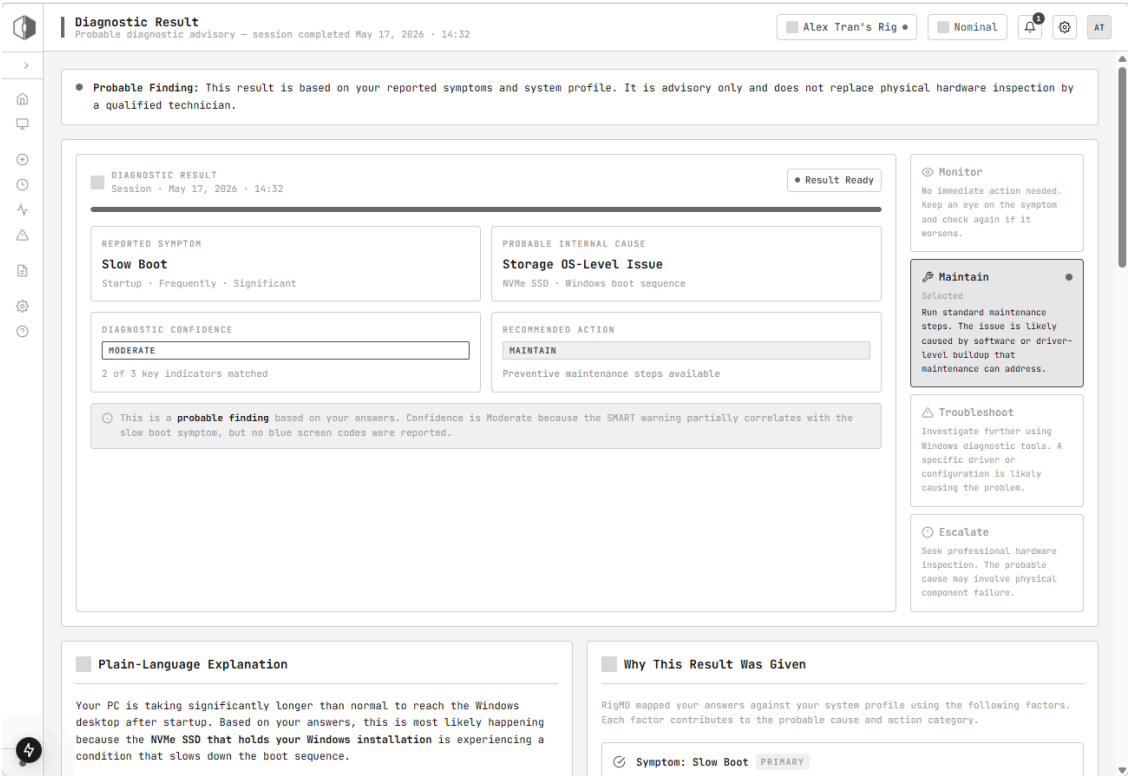
Use Case Diagram



Activity Diagram



Wireframe



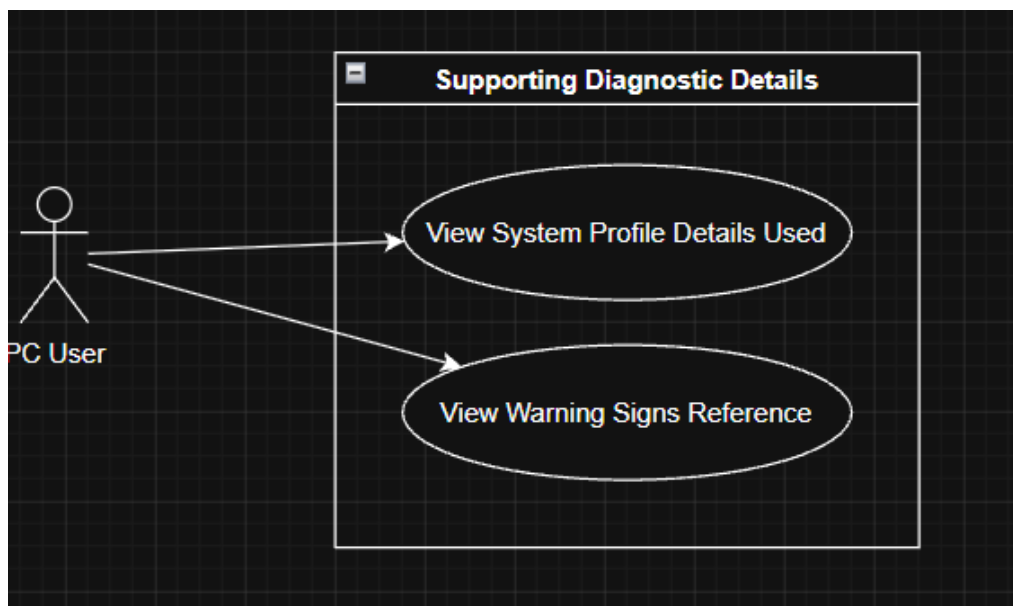
Module 3.2: Supporting Diagnostic Details

Use Case Description

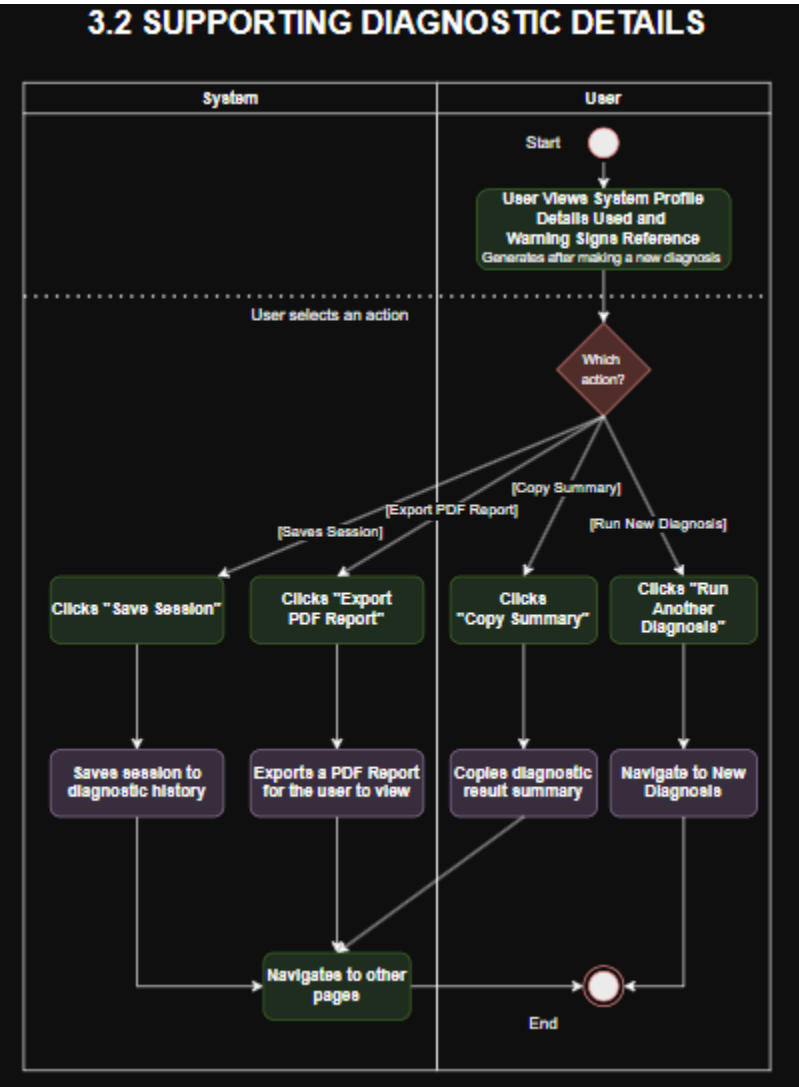
Use Case ID	UC-3.2
Use Case Name	Supporting Diagnostic Details
Actor	PC User
Precondition	The user is viewing a completed Diagnostic Result screen.

Main Flow	The user scrolls down to the Supporting Diagnostic Details section. The system displays the specific system profile fields that directly influenced the result alongside a Warning Signs Reference panel. The user reviews these panels to see exactly which hardware specs and observable indicators were matched during the session.
Alternative Flow	If a profile field was undetected, it is marked as unverified and visually excluded from the reasoning factors.
Postcondition	The user sees the exact data points that triggered the diagnostic result.
Non-functional Requirement	The details panel must accurately reflect the locally stored system profile without modification.

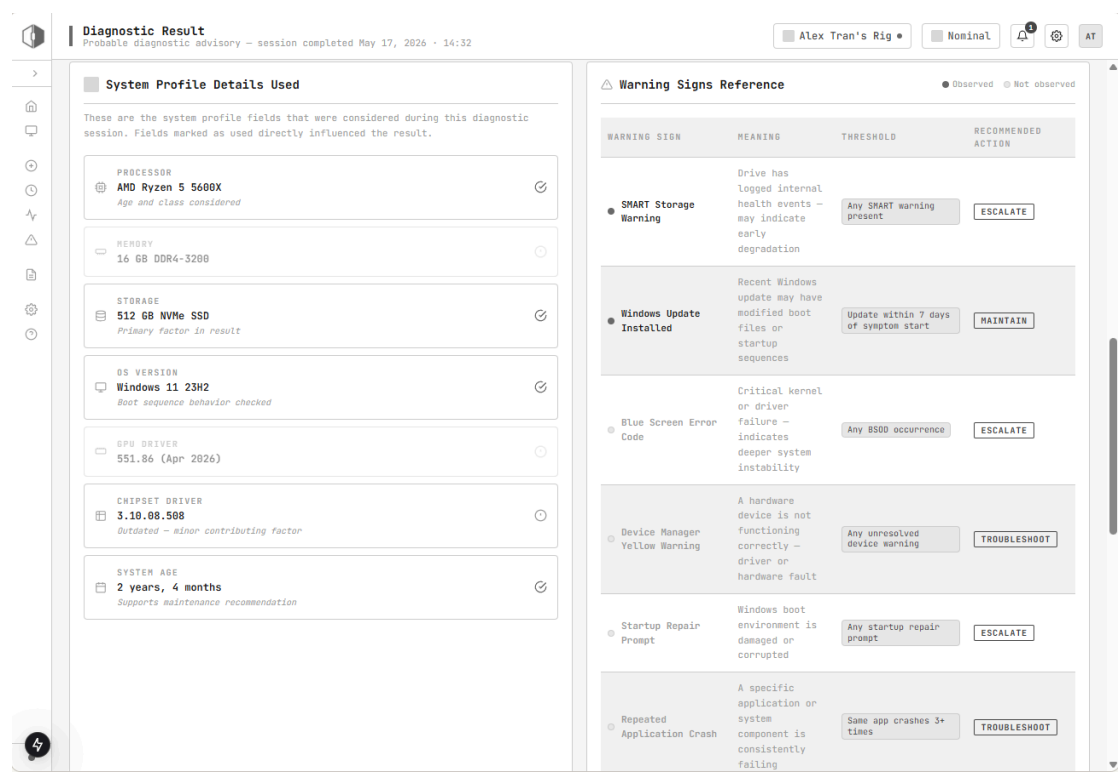
Use Case Diagram



Activity Diagram



Wireframe



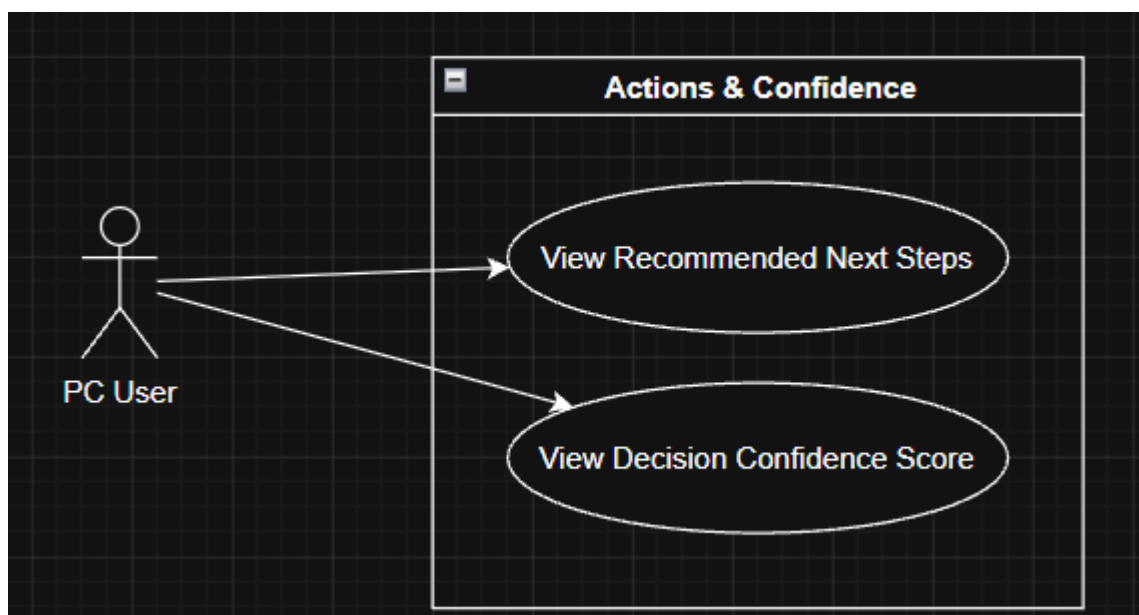
Module 3.3: Recommended Actions and Decision Confidence

Use Case Description

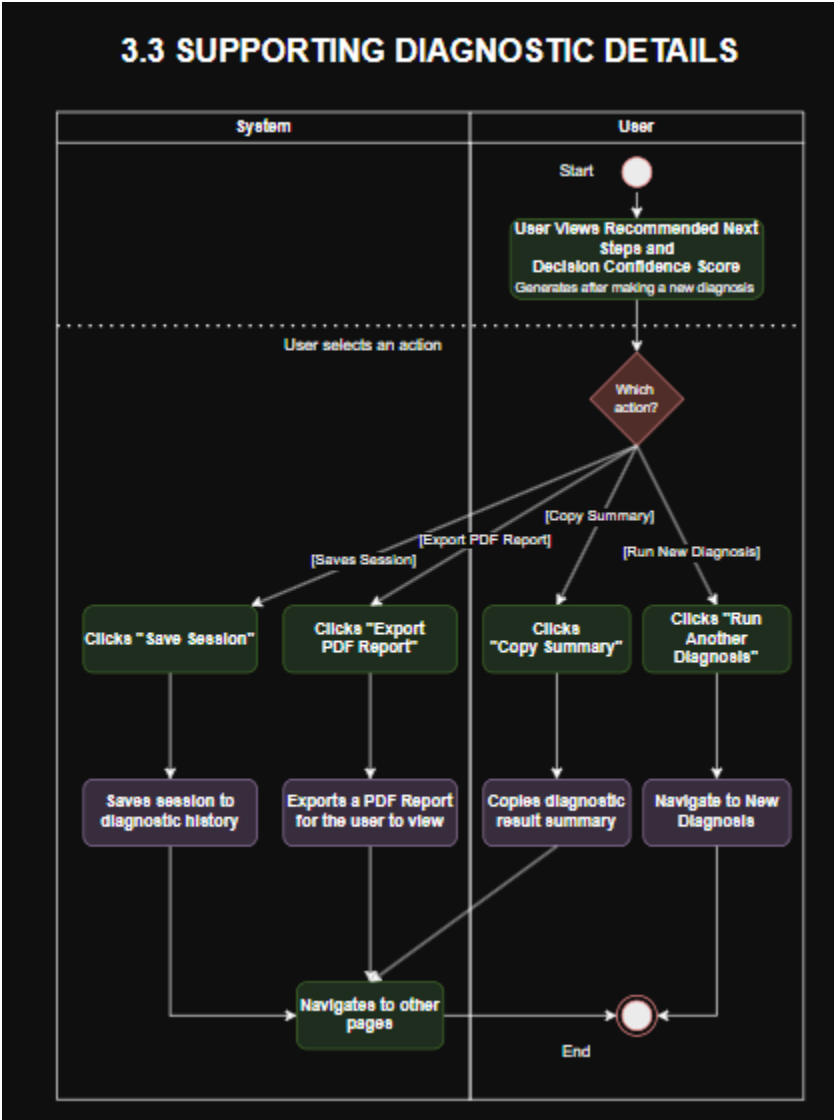
Use Case ID	UC-3.3
Use Case Name	Recommended Actions and Decision Confidence
Actor	PC User
Precondition	The user is viewing the lower section of a completed Diagnostic Result screen.

Main Flow	The user reviews the Recommended Next Steps section. The system displays a prioritized sequence of actions to address the PC issue, complete with priority tags like "DO FIRST". The user then views the Decision Confidence panel to review how much their understanding of the problem has improved since starting the session.
Alternative Flow	If the system classifies the issue strictly as "Monitor," the Next Steps section displays observation instructions instead of active repair tasks.
Postcondition	The user is aware of the necessary steps to resolve or mitigate the issue.
Non-functional Requirement	Recommendations must be written in non-technical plain language.

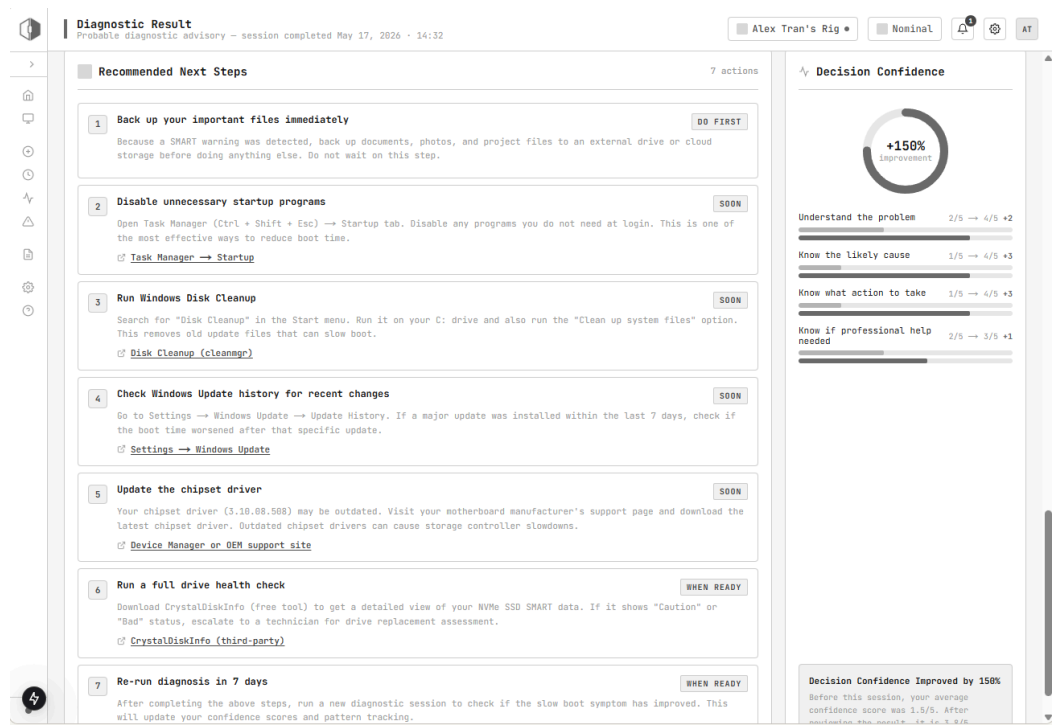
Use Case Diagram



Activity Diagram



Wireframe



Module 4: Advisory Action Module

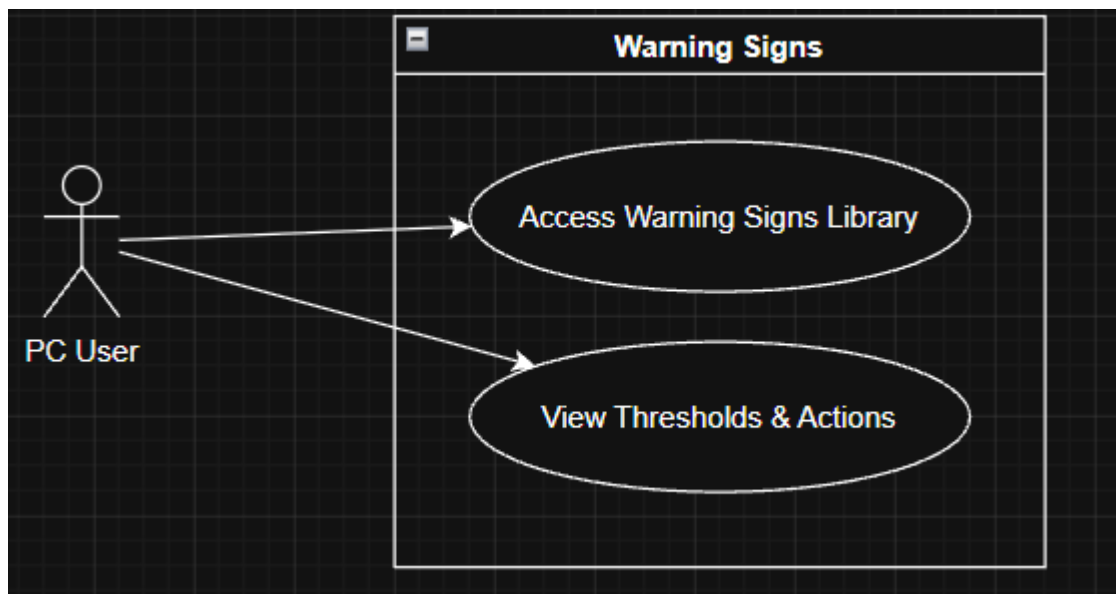
Module 4.1: Warning Signs

Use Case Description

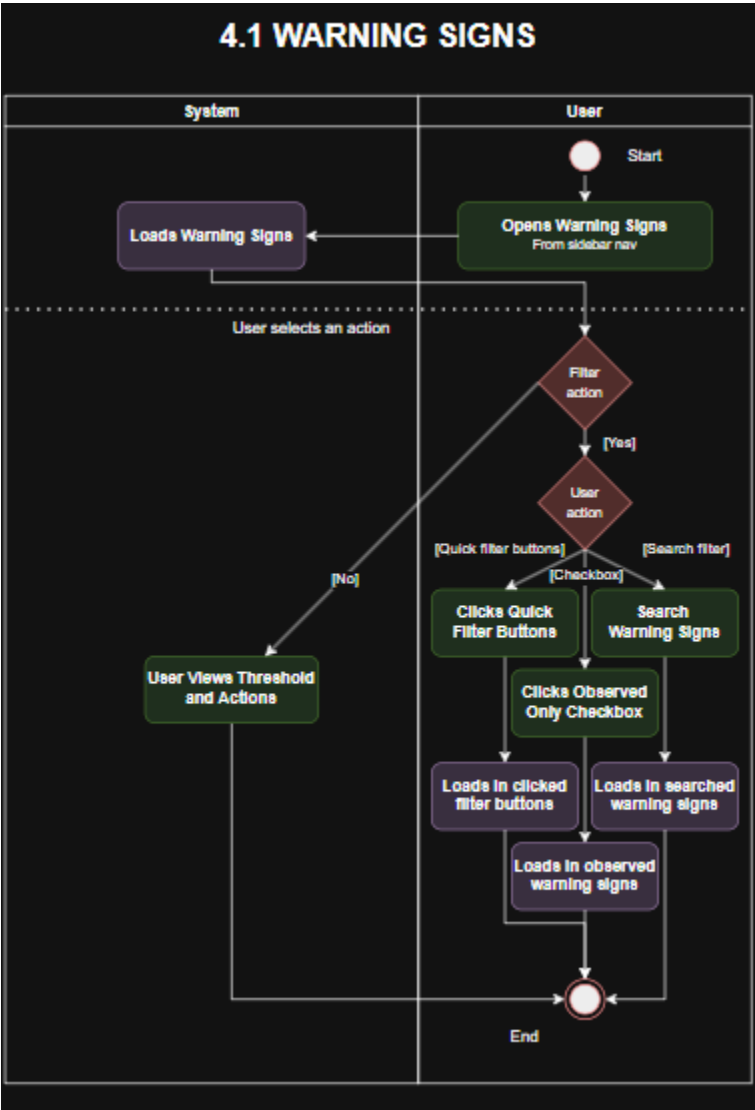
Use Case ID	UC-4.1
Use Case Name	Warning Signs Reference
Actor	PC User
Precondition	The user accesses the main navigation menu.

Main Flow	The user selects the Warning Signs section from the sidebar. The system displays a standalone reference library of observable PC warning indicators. The user reviews the meaning, critical thresholds, and recommended actions for various hardware and software symptoms without needing to complete a full diagnostic session.
Alternative Flow	The user can filter the library by specific categories such as System Crash or Storage using the provided category pills.
Postcondition	The user gains knowledge about system health indicators and escalation points.
Non-functional Requirement	The warning signs table must render completely within 2 seconds.

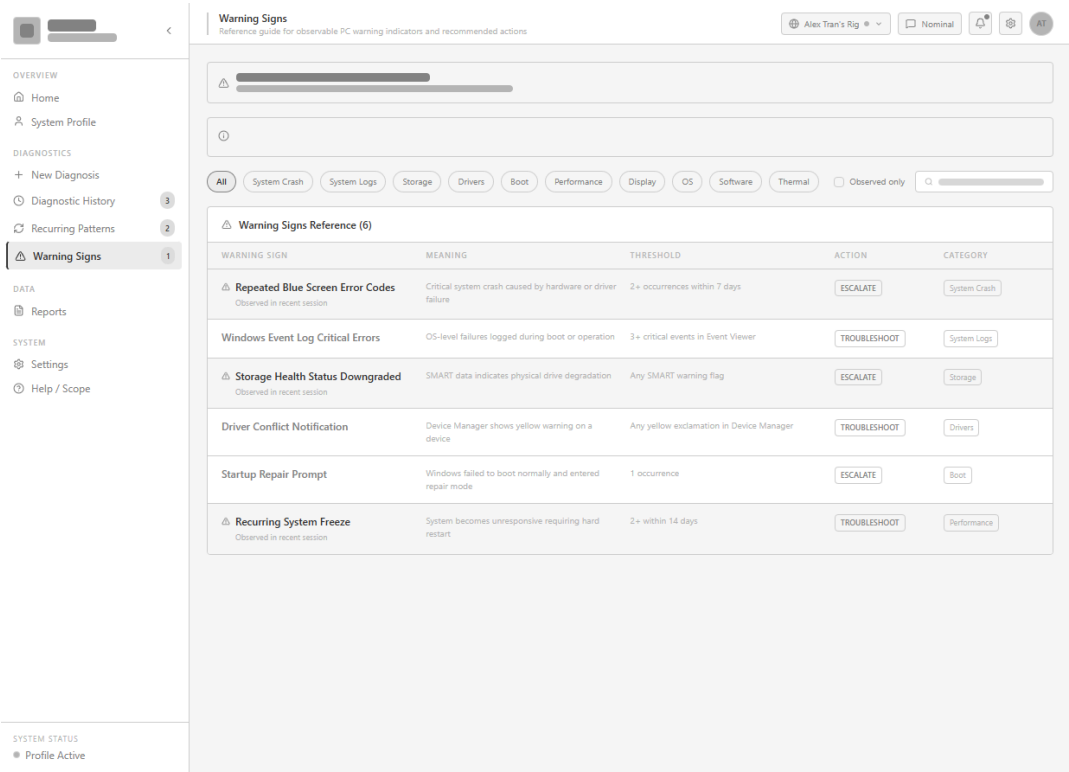
Use Case Diagram



Activity Diagram



Wireframe



Module 5: Transparent Result Dashboard and Report Output Module

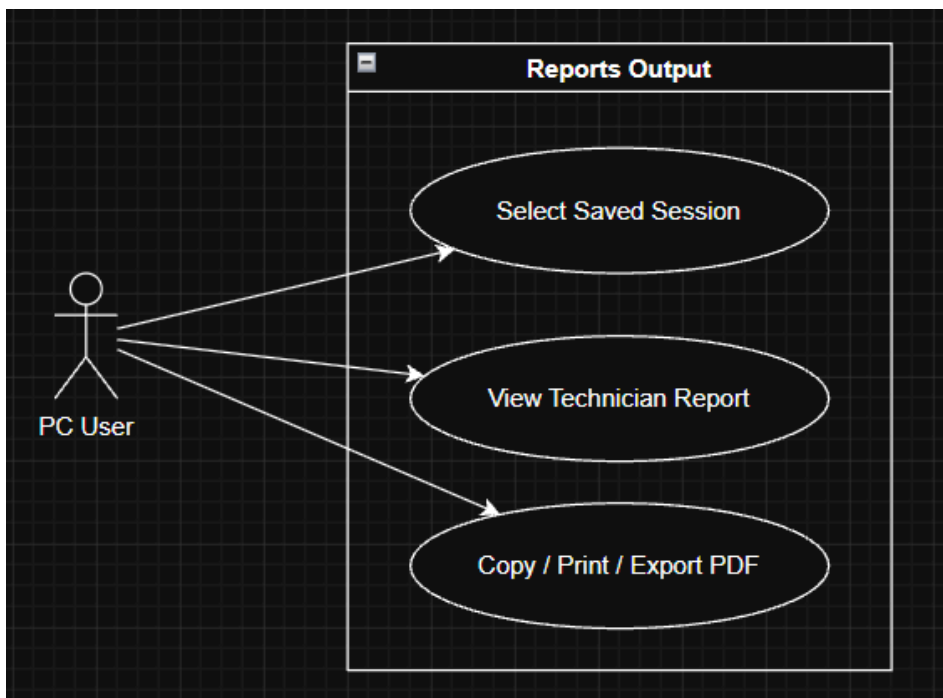
Module 5.1: Reports

Use Case Description

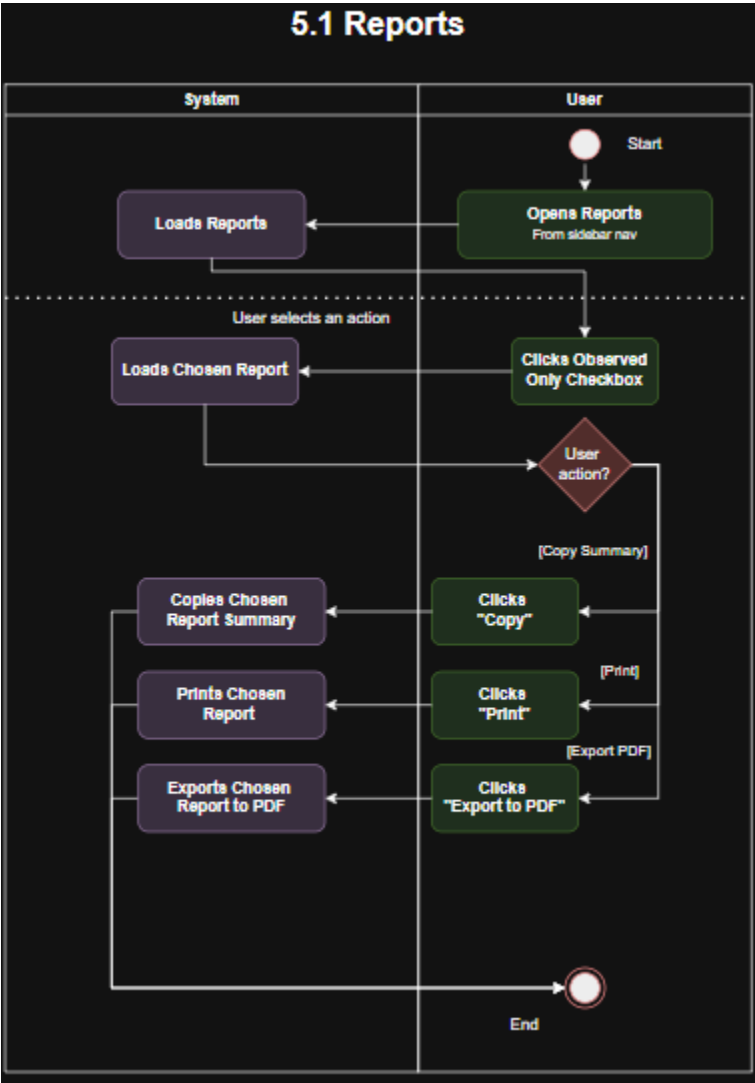
Use Case ID	UC-5.1
Use Case Name	Report Output Generation
Actor	PC User
Precondition	The user navigates to the Reports section.

Main Flow	The user selects a previously completed diagnostic session from the dropdown menu. The system compiles and displays a technician-ready report containing the session summary, profile snapshot, and symptom answers. The user utilizes the provided interface buttons to copy the data, send it to a printer, or export it as a PDF document.
Alternative Flow	If no sessions are saved locally, the system disables the export options and prompts the user to run their first diagnosis.
Postcondition	A shareable report is successfully generated and exported.
Non-functional Requirement	The PDF export must be generated and ready to save within 5 seconds.

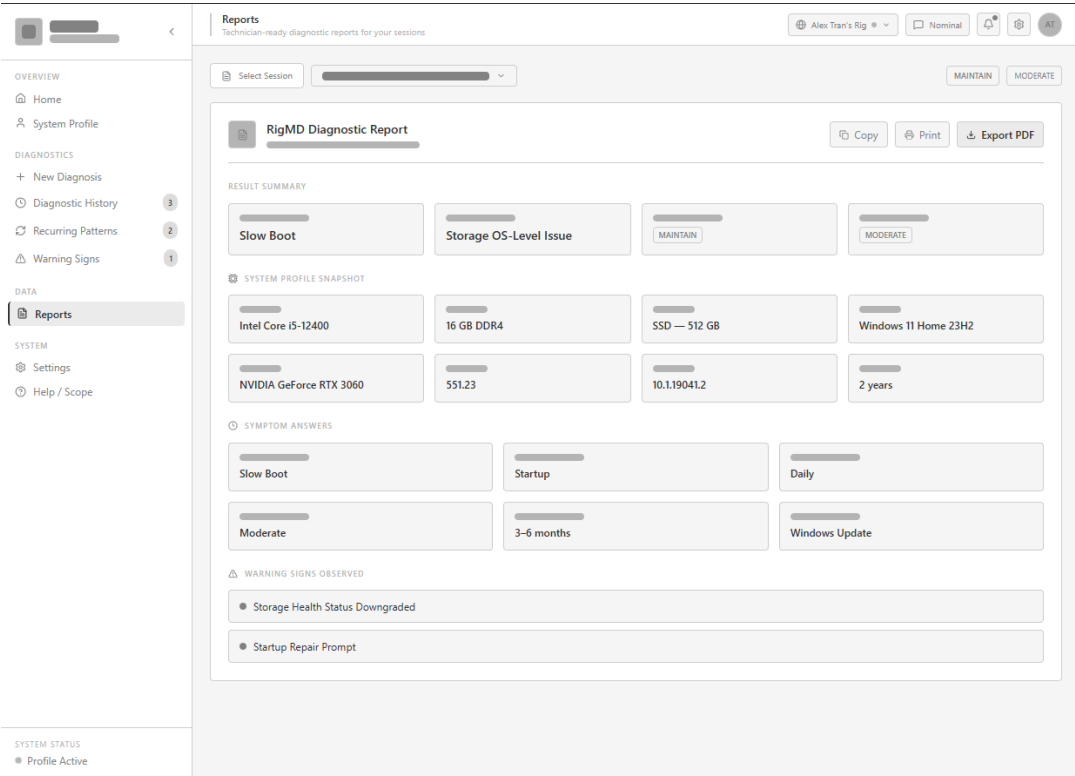
Use Case Diagram



Activity Diagram



Wireframe



Module 6: Session History and Recurring Pattern Detection Module

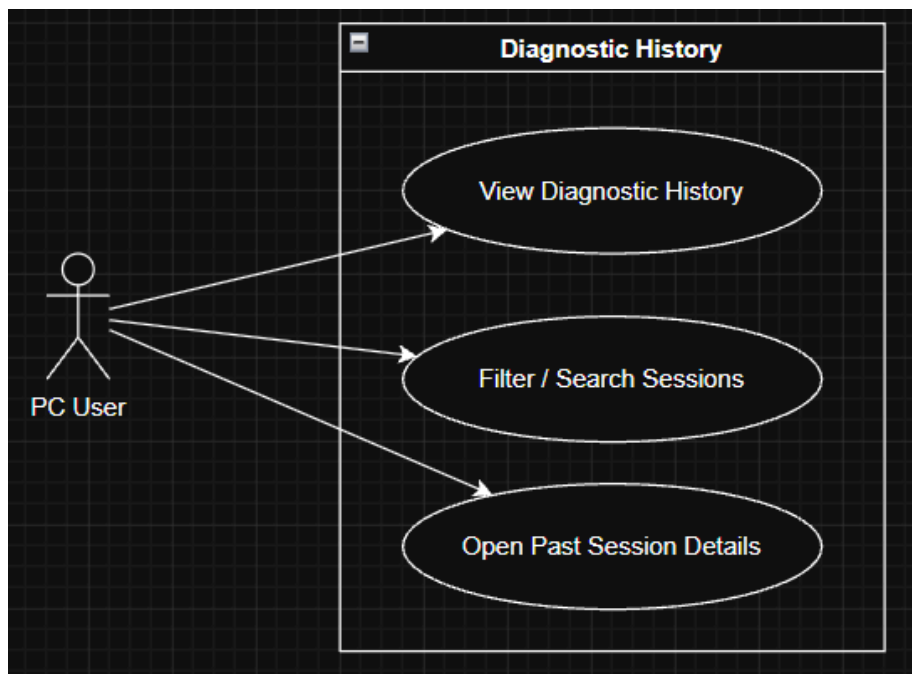
Module 6.1: Diagnostic History

Use Case Description

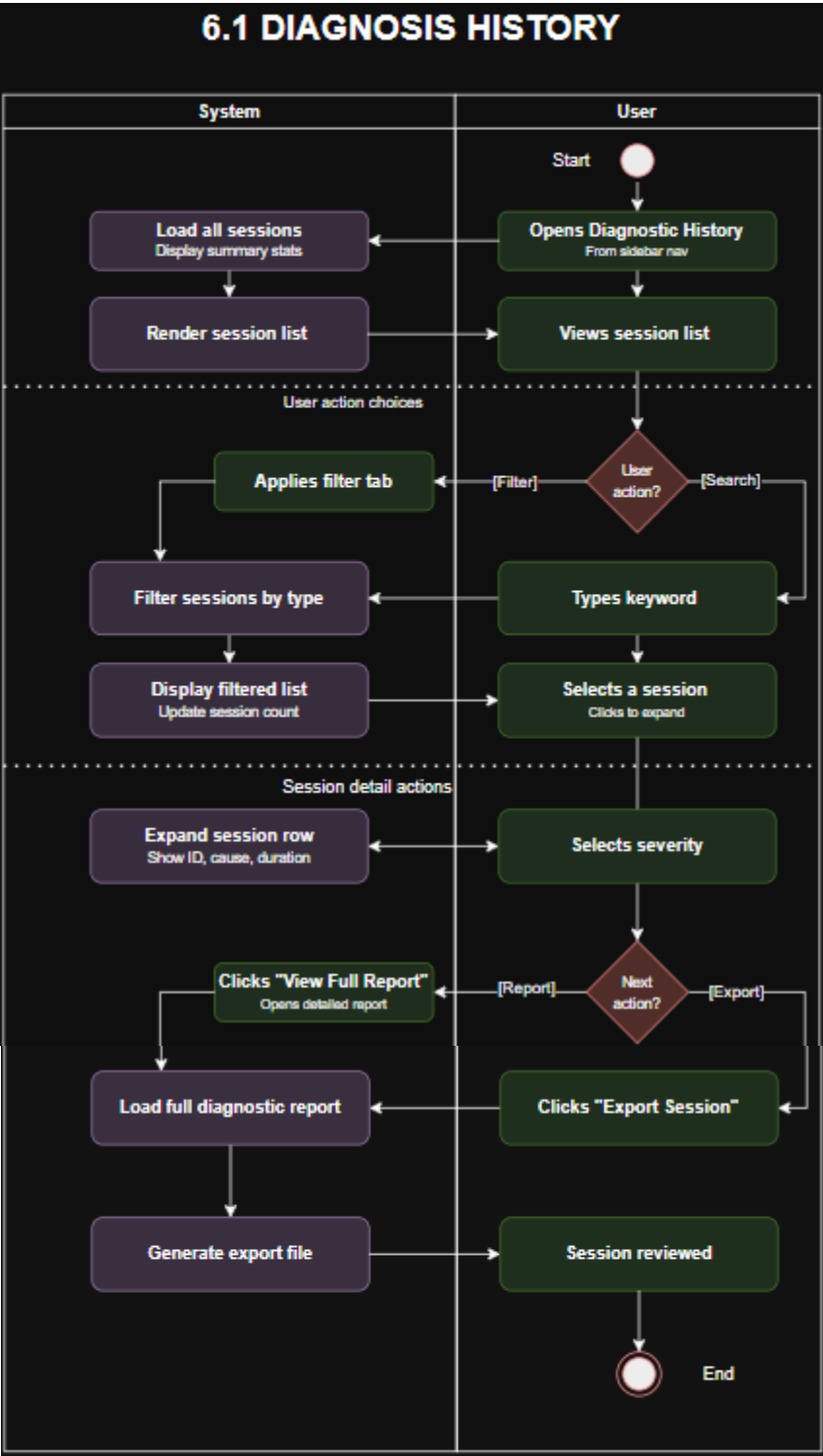
Use Case ID	UC-6.1
Use Case Name	Diagnostic History Management
Actor	PC User
Precondition	The user has completed at least one diagnostic session.

Main Flow	The user navigates to the Diagnostic History section. The system displays a chronological list of all saved diagnostic sessions and their specific action outcomes. The user applies category filters or searches for specific issues. The user clicks on a past session to open and view its specific details and confidence scores.
Alternative Flow	If the local history database is empty, the system displays a placeholder prompt instructing the user to start a new diagnosis.
Postcondition	The user successfully retrieves and reviews their historical PC health data.
Non-functional Requirement	The history list must load up to 50 local records in under 3 seconds.

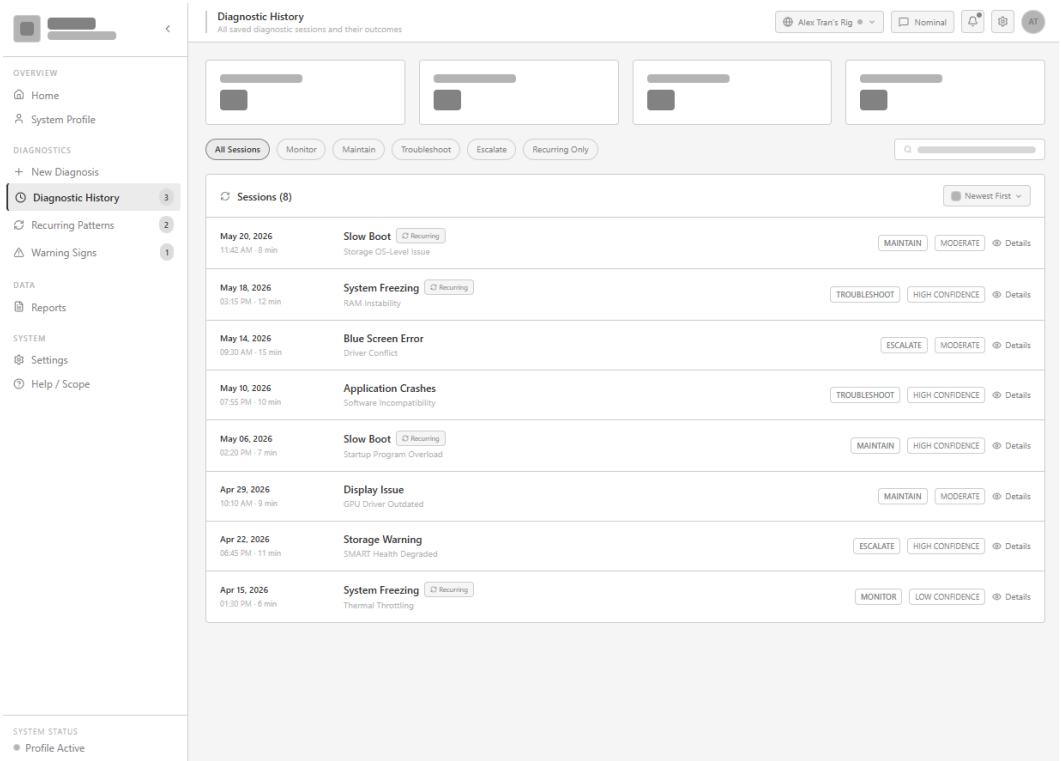
Use Case Diagram



Activity Diagram



Wireframe



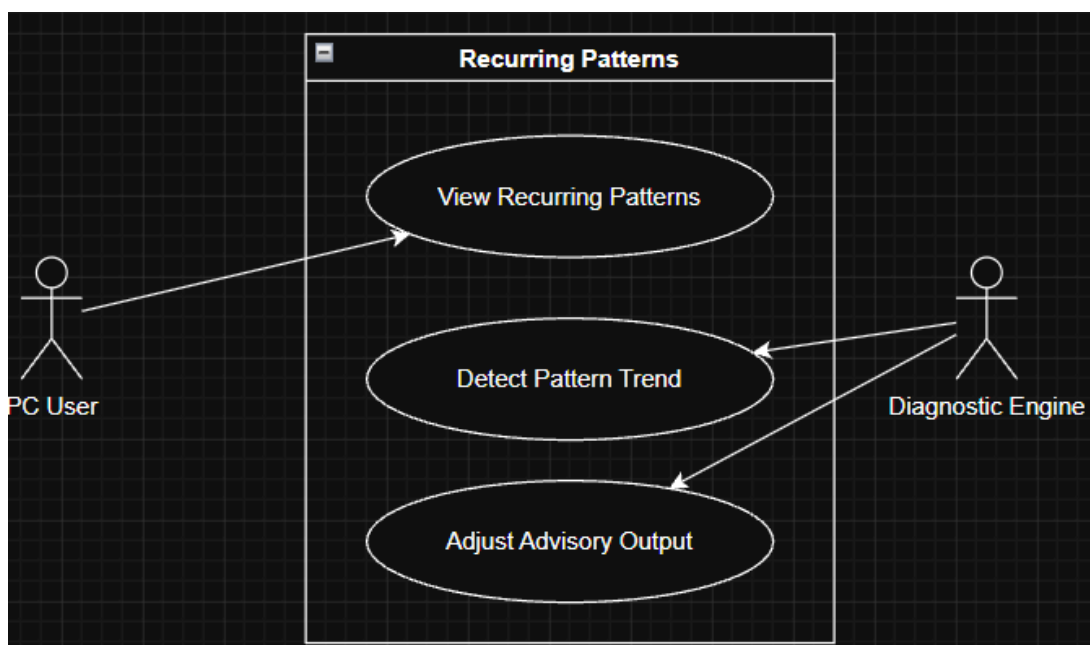
Module 6.2: Recurring Patterns

Use Case Description

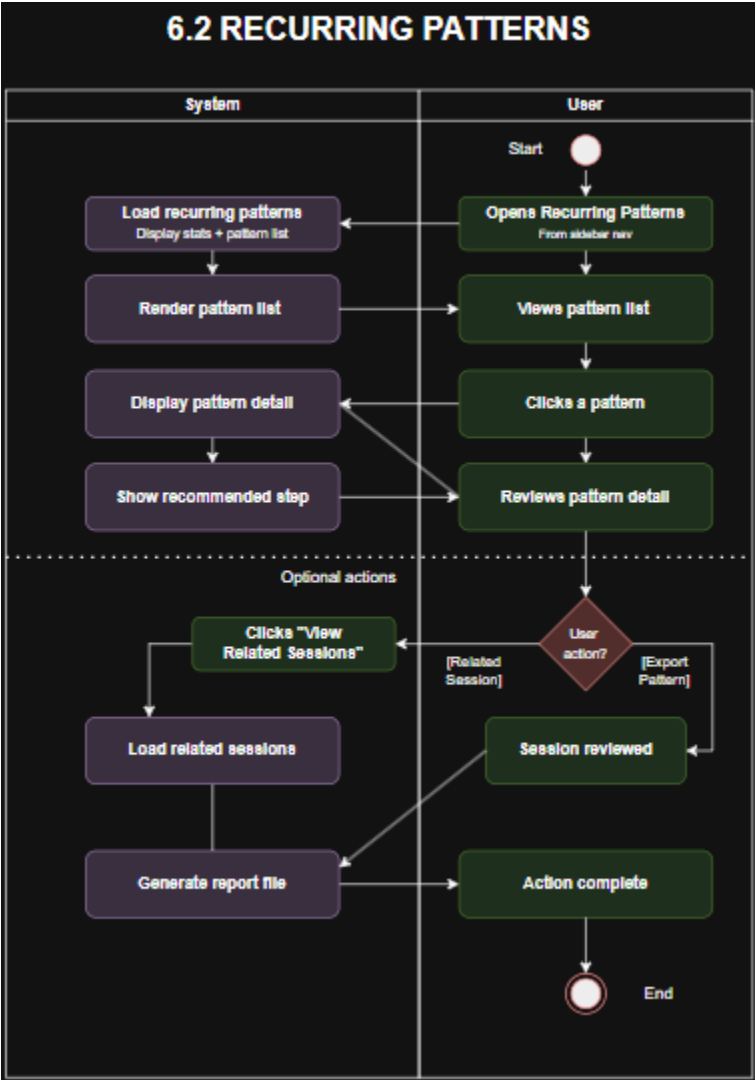
Use Case ID	UC-6.2
Use Case Name	Recurring Pattern Detection
Actor	PC User, Diagnostic Engine
Precondition	The user has completed multiple diagnostic sessions over time.

Main Flow	The user navigates to the Recurring Patterns section. The system retrieves and displays comparisons of repeated symptoms and cause categories across past sessions. The user checks the occurrence count, first seen date, and trend label to determine if the issue is stable, improving, or worsening. The Diagnostic Engine automatically tracks these trends in the background to appropriately adjust future advisories.
Alternative Flow	If no patterns are detected, the system displays a message indicating that the PC currently shows no repeated issues.
Postcondition	The user understands long-term health trends affecting their PC, and the system is primed to escalate worsening conditions.
Non-functional Requirement	Pattern detection must process local history data securely without requiring cloud computation.

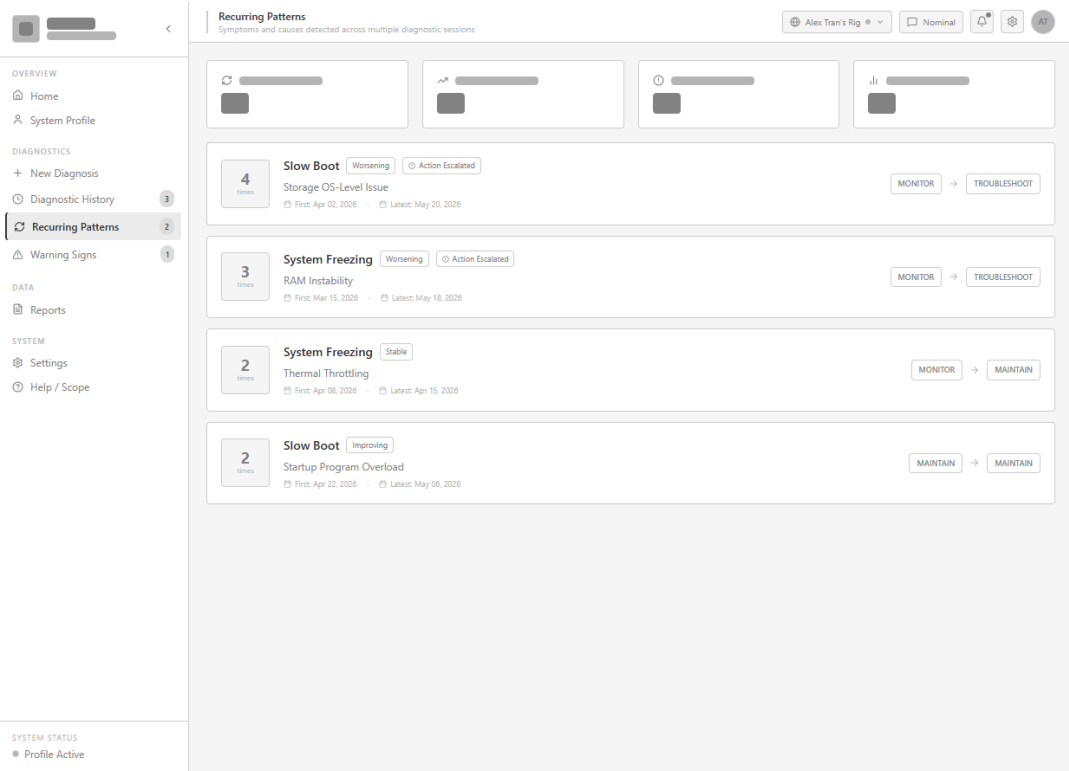
Use Case Diagram



Activity Diagram



Wireframe



3.3 Data Requirements

The following local data must be stored or processed by the application:

Data Item	Required Content
System Profile Record	CPU model, RAM capacity, storage type and capacity, operating system version, GPU driver version, chipset driver version, system age, detection confidence, and last scan timestamp.
Diagnostic Session Record	Session ID, timestamp, selected symptom type, affected activity, frequency, severity, duration, recent changes, system state, warning signs, and reviewed answers.
Diagnostic Output Record	Probable internal cause category, advisory action category, confidence label, reasoning factors, explanation, recommended next steps, and warning signs reference.
Recurring Pattern Record	Repeated symptom type or internal cause category, occurrence count, first seen date,

	latest date, trend label, previous action category, and current action category.
Report Record	Selected session summary, system profile snapshot, symptom answers, warning signs observed, result summary, and export metadata.

All records must be stored locally. The prototype must not require user accounts or remote database synchronization.

3.4 Non-Functional Requirements

Performance

Requirement	Description
Startup Time	The application shall launch and display the main dashboard within 5 seconds on a device meeting the minimum requirements.
Questionnaire Responsiveness	Each questionnaire step shall transition within 500 milliseconds after a user response is selected.
Rule-Based Result Time	The application shall display the diagnostic result within 3 seconds when using offline or rule-based-only mode.
AI-Assisted Result Time	The application shall display the diagnostic result within 10 seconds after submission when AI-assisted explanation generation is available and the network connection is stable.
Session Save Speed	Completed session records shall be saved to local storage within 2 seconds of result generation.

History Load Time	The diagnostic history dashboard shall load and display saved session records within 3 seconds for at least 50 saved sessions.
Report Generation Time	A selected session report shall be prepared for viewing, copying, printing, or PDF export within 5 seconds under normal operation.

Security and Privacy

Requirement	Description
Local Data Storage	All system profile and session records shall be stored locally on the user's device.
No User Authentication Required	The prototype shall not require user accounts, login, registration, or cloud profile synchronization.
AI API Data Limitation	AI API requests shall include only structured symptom data, system profile categories or values needed for explanation, and the classified output. Personally identifiable information shall not be sent.
Encrypted Communication	All AI API communication shall use HTTPS.
Input Validation	All symptom responses, profile values, and local data records shall be validated before processing. Invalid or incomplete data shall produce clear user-facing messages.
Data Integrity	The system shall detect corrupted or unreadable local data files and prevent the application from entering an inconsistent state.

Reliability

Requirement	Description
-------------	-------------

Offline Availability	Core functions shall operate without internet access, including profile viewing, symptom intake, rule-based interpretation, advisory classification, result display, session history, and recurring pattern detection.
AI Fault Tolerance	If the AI API is unavailable, the system shall use fallback explanation templates and inform the user that AI-assisted explanation is unavailable.
Data Persistence	System profile and session history shall persist across application restarts under normal operation.
Error Handling	The system shall provide plain-language messages for file I/O errors, API timeout, invalid input, corrupted data, and unavailable profile data.
Safety Boundary	The system shall avoid unsupported claims and shall recommend professional inspection when symptoms exceed software-layer resolution.

Usability

Requirement	Description
Plain-Language Interface	The interface shall present symptom questions, explanations, and recommendations in terms understandable to non-technical users.
Guided Workflow	The diagnostic process shall use step-by-step controlled questions rather than requiring free-text diagnosis from the user.
Transparency	The result dashboard shall show the reasoning factors, system profile details used, warning signs, and next steps that support the recommendation.

Consistency	The desktop interface shall use consistent navigation, labels, filters, buttons, and card layouts across all screens.
Evaluation Target	The system shall achieve a mean SUS score of at least 75 in usability testing with at least 10 target users.
Decision Confidence Target	The system shall demonstrate at least 25% improvement in composite user decision confidence ratings from pre-session to post-session evaluation.

Diagnostic Accuracy and Validation

Requirement	Description
Expert Validation Threshold	The diagnostic and recommendation outputs shall achieve at least 80% expert-validated acceptance across at least 20 controlled diagnostic scenarios.
Scenario Coverage	Controlled scenarios shall cover slow boot, freezing, repeated blue screen errors, driver conflict behavior, storage warning indicators, application crashes after updates, no-display behavior after a driver change, and basic no-power triage.
Recommendation Relevance	Expert evaluators shall assess whether the advisory action category is appropriate for the given symptom pattern and system profile.
Explanation Clarity	Expert evaluators shall assess whether the explanation accurately connects symptoms, profile data, probable internal cause, and recommended action.

4. Validation and Evaluation Requirements

The system shall be evaluated using expert validation, usability testing, and decision confidence measurement.

Evaluation Area	Requirement
Expert Validation	At least two hardware-knowledgeable evaluators shall review the system output across a minimum of 20 controlled scenarios. Each scenario shall include a sample system profile, Windows-observable symptoms, expected internal cause category, and expected advisory action.
Usability Testing	At least 10 target users shall complete a session with the prototype and answer the System Usability Scale after using the system. The target mean SUS score is at least 75.
Decision Confidence Survey	A structured Likert-scale survey shall be administered before and after each session to measure confidence in understanding the problem, identifying the likely cause, understanding the recommendation, and deciding on the next action. The target improvement is at least 25%.
Qualitative Feedback	Short post-test feedback shall be collected around explanation clarity, trust in recommendation, ease of guided workflow, usefulness of warning signs, and value of session history.